RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

# QUEUES

## Types and Implementation

Simple • Circular • Double-Ended • Priority

**Implementation using Arrays and Linked Lists**

**Data Structures and Applications**

Based on E. Horowitz, S. Sahni and S. Anderson-Freed
Fundamentals of Data Structures in C

# Scope of the Topic

Understand the four queue types: simple, circular, double-ended and priority queues.

Represent queues using a one-dimensional array and identify the role of front and rear.

Implement simple and circular queues while handling overflow and underflow conditions.

Represent a queue using linked nodes and front/rear pointers.

Compare array-based and linked-list implementations in terms of storage and operations.

Recognize when a particular queue type is appropriate for an application.

# Queue Types: Overview

### Simple Queue

FIFO; insertion at rear, deletion at front.

### Circular Queue

Rear wraps around to reuse free array positions.

### Double-Ended Queue

Insertion and deletion are possible at both ends.

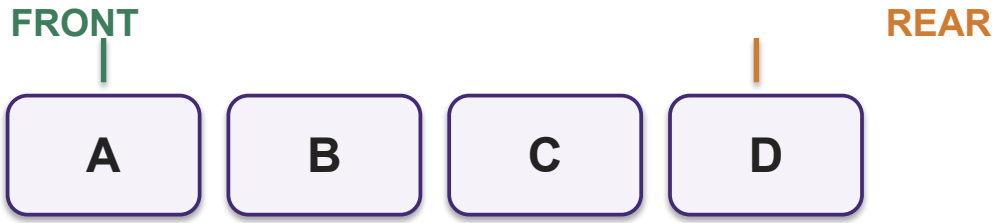
### Priority Queue

Deletion/service is determined by priority.

**The queue type changes the rule used to determine where insertion/deletion can occur.**

# Simple Queue

**A simple queue follows strict FIFO ordering.**




### Insertion

Add the new element at rear.

### Deletion

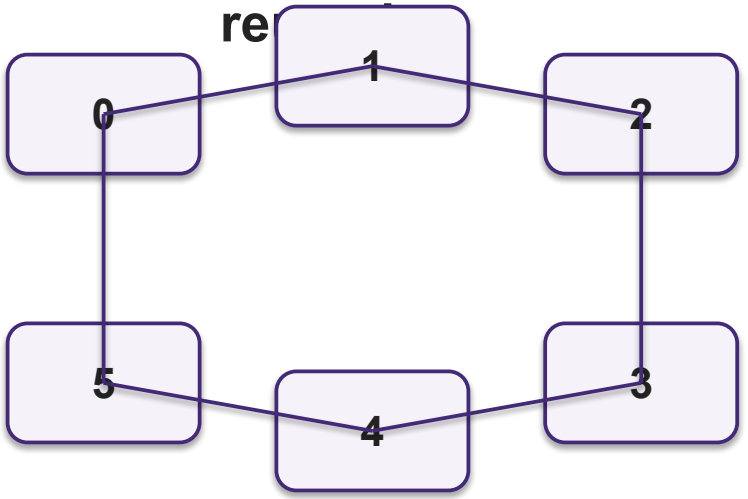
Remove the element at front.

### Ordering

First inserted → first removed.

# Circular Queue

**A circular queue treats the array as a circular sequence so that freed positions can be re**

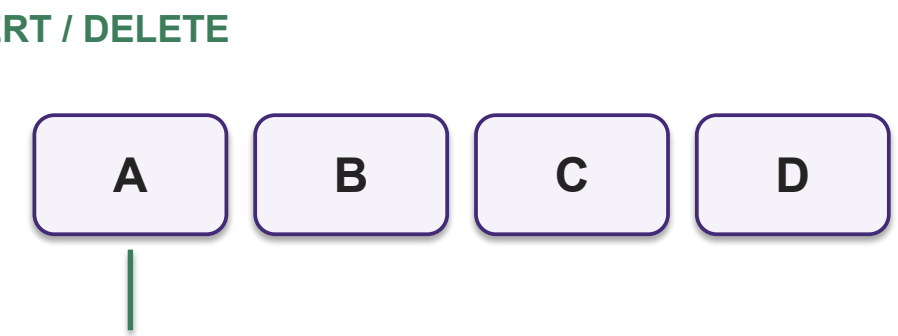



When rear reaches the last index, the next position can wrap to index 0 if it is available.

**Circular update: next = (current + 1) mod MAX_QUEUE_SIZE**

# Double-Ended Queue (Deque)

**A deque permits insertion and deletion at either end.**




### Input-Restricted Deque

Insertion is restricted to one end; deletion can occur at either end.

### Output-Restricted Deque

Deletion is restricted to one end; insertion can occur at either end.

# Priority Queue

**A priority queue serves elements according to priority rather than simply according to arrival time.**

| J1<br>P=3 | J2<br>P=1 | J3<br>P=2 | J4<br>P=1 |
|---|---|---|---|

If smaller number means higher priority, J2 and J4 are served before J3 and J1.

### Service rule

Priority determines the next element.

### Tie handling

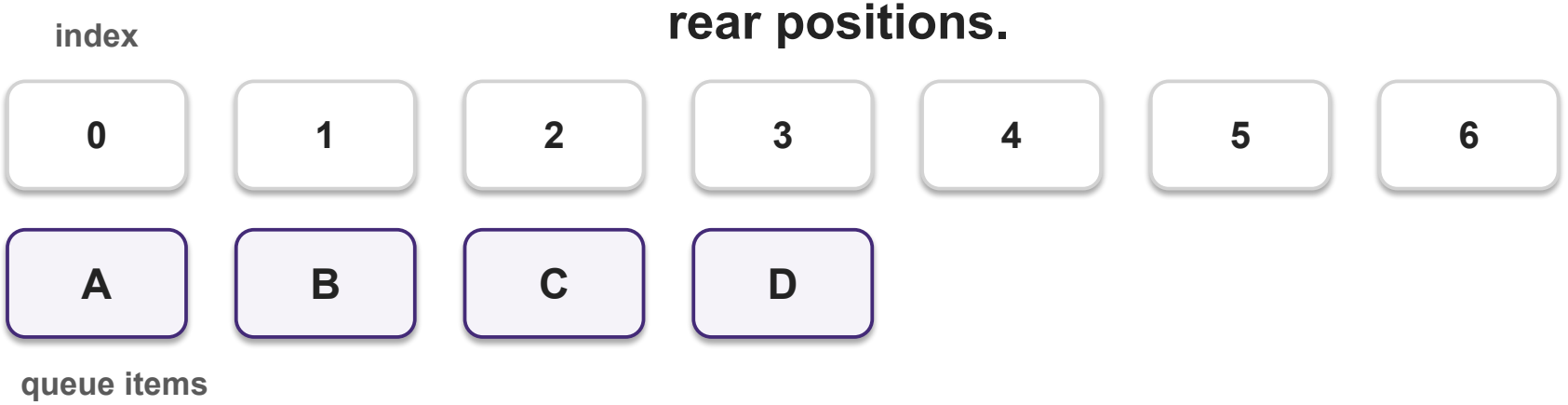
Equal priorities may be handled FIFO.

### Applications

Scheduling, simulations and service systems.

# Queue Representation Using an Array

**The sequential representation uses a one-dimensional array together with front and rear positions.**




front identifies the element to be deleted.

rear identifies the position for the next insertion.

# Array Implementation: Simple Queue

**The array implementation maintains two indices: front and rear.**

```
Queue {
    KeyType queue[MAX];
    int front;
    int rear;
};

Initialize:
    front = 0;
    rear  = -1;
```

### Enqueue / Add

Increment rear and place the new item in queue[rear], after checking for full condition.

### Dequeue / Delete

Read queue[front], then advance front, after checking for empty condition.

The exact empty/full conventions depend on the chosen array representation.

# Simple Queue: Enqueue Operation

```
Add(item)
{
    if (IsFull())
        QueueFull();
    else {
        rear++;
        queue[rear] = item;
    }
}
```

**Step 1**

Check whether the queue is full.

**Step 2**

Advance rear and store item at queue[rear].

**Insertion always occurs at the rear in a simple queue.**

# Simple Queue: Dequeue Operation

```
Delete(item)
{
    if (IsEmpty())
        QueueEmpty();
    else {
        item = queue[front];
        front++;
    }
}
```

**Step 1**

Check whether the queue is empty.

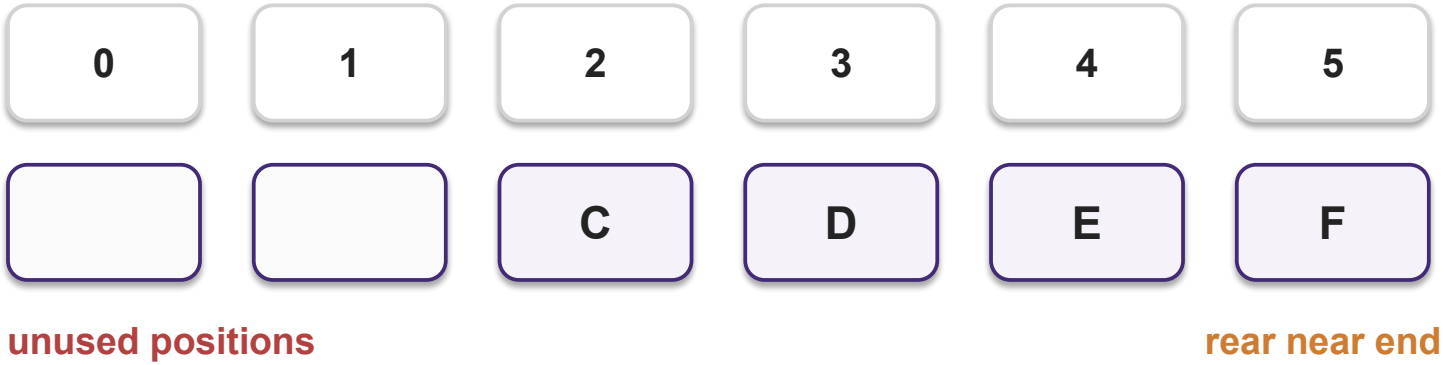
**Step 2**

Return the front item and advance front.

**Deletion always occurs at the front in a simple queue.**

# Why a Linear Array Queue Has a Limitation

**After deletions, free positions can appear at the beginning while rear moves toward the last array position.**




The intuitive array implementation may require elements to move to reclaim unused space.

In the worst case, such shifting can take O(n) time.

Circular queues remove the need for this repeated shifting.

# Circular Queue: Array Implementation

```
Initialize:
    front = rear = 0;

Next position:
    next = (position + 1) % MAX;

Enqueue:
    newRear = (rear + 1) % MAX;
    queue[newRear] = item;
    rear = newRear;
```

## Wrap-around

After the last array position, the next position is the first array position.

## Full condition

One common convention is to reserve one position so that the full and empty states remain distinguishable.

## Alternative

A flag/count can also distinguish full from empty.

# Circular Queue: Dequeue Operation

```
Delete(item)
{
    if (IsEmpty())
        QueueEmpty();
    else {
        front = (front + 1) % MAX;
        item = queue[front];
    }
}
```

### 1. Empty check

Do not delete when the queue has no elements.

### 2. Circular advance

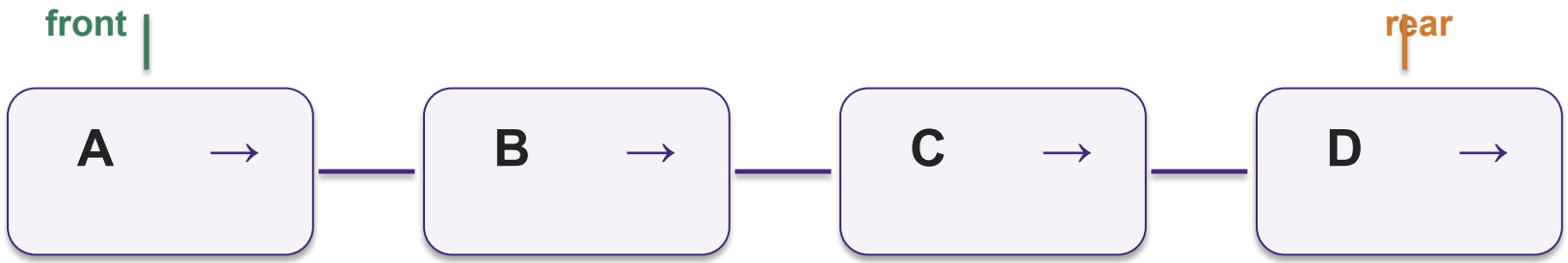
Move front to its next circular position.

### 3. Return item

The new front identifies the item to remove under this convention.

# Queue Representation Using a Linked List

**A linked queue uses dynamically allocated nodes rather than a fixed-size array.**




### Node

Contains the data item and a link to the next node.

### Front pointer

Identifies the first node to be deleted.

### Rear pointer

Identifies the last node where a new node is linked.

# Linked Queue: Node Structure

```
typedef struct node {
    KeyType data;
    struct node *link;
} Node;

Node *front = NULL;
Node *rear  = NULL;
```

Each node stores one queue element.

link points to the next node.

front points to the first node.

rear points to the last node.

For an empty queue, both front and rear are NULL.

**Unlike a fixed array, the linked representation grows and shrinks dynamically as nodes are allocated and freed.**

# Linked Queue: Enqueue

```
Add(item)
{
    new = allocate Node;
    new->data = item;
    new->link = NULL;

    if (rear == NULL)
        front = rear = new;
    else {
        rear->link = new;
        rear = new;
    }
}
```

### Case 1: Empty queue

The first node becomes both front and rear.

### Case 2: Non-empty

Link the new node after rear and move rear to the new node.

### Time

With a rear pointer: O(1).

# Linked Queue: Dequeue

```
Delete(item)
{
    if (front == NULL)
        QueueEmpty();
    else {
        temp = front;
        item = temp->data;
        front = front->link;
        free(temp);

        if (front == NULL)
            rear = NULL;
    }
}
```

### Empty queue

Report underflow / QueueEmpty.

### Normal deletion

Remove the first node and advance front.

### Last node

If front becomes NULL, reset rear to NULL.

# Array Queue vs Linked Queue

| Aspect | Array implementation | Linked-list implementation |
|---|---|---|
| **Storage** | Fixed capacity (unless dynamic array is used) | Dynamic; grows as nodes are allocated |
| **Memory layout** | Contiguous | Non-contiguous nodes linked by pointers |
| **Overflow** | When capacity is reached | Only when memory allocation fails |
| **Space reuse** | Circular queue reuses positions | Freed nodes can be returned to memory |
| **Pointers / indices** | front and rear indices | front and rear node pointers |
| **Enqueue / Dequeue** | O(1) with suitable representation | O(1) with front and rear pointers |

# Implementing a Deque

**A deque can be implemented with an array or a linked structure, depending on the required operations.**

## Array-based deque

Use front and rear indices.

A circular array is particularly suitable because positions can be reused at both ends.

Operations: insertFront, insertRear, deleteFront, deleteRear.

## Linked deque

Use a doubly linked list so that both ends can be updated efficiently.

Maintain pointers to the first and last nodes.

Operations at either end can be O(1).

## Choice

Array: compact contiguous storage and predictable capacity.

Linked: dynamic size and flexible memory usage.

The choice depends on application constraints.

# Priority Queue Implementation Choices

**Priority queues can use different underlying representations; the representation affects operation cost**

## Unordered Array / List

Insertion is simple because the new item can be placed at the available position.

Finding the highest-priority item for deletion may require a scan.

## Ordered Array / List

The elements are maintained in priority order.

Deletion of the highest-priority element is easy, but insertion may require shifting or searching.

## Linked Representation

Nodes can be maintained in sorted or unsorted order.

Pointers replace contiguous array storage, while the priority ordering rule remains the same.

**For advanced priority-queue structures, specialized implementations can provide improved performance.**

# Choosing the Appropriate Queue

## Simple Queue

Need strict FIFO service and straightforward operations.

## Circular Queue

Need fixed-size array storage with efficient reuse of released positions.

## Deque

Need insertion/deletion at both ends.

## Priority Queue

Need service according to priority rather than pure arrival order.

# Worked Comparison

**Suppose A, B, C and D arrive in that order.**

## Simple Queue

Removal order:
A → B → C → D

Strict FIFO.

## Circular Queue

Same logical FIFO order, but storage positions wrap around in the array.

## Deque

Removal may be from either end.

Order depends on the chosen operations.

## Priority Queue

Removal order depends on priority.

Arrival order may not determine service order.

**Key distinction: Simple/Circular queues preserve FIFO; Deques change end-access rules; Priority queues change the service-order rule.**

# Applications of Queue Types

## Simple Queue

Printer requests, basic scheduling, waiting-line simulations.

## Circular Queue

Buffers, streaming data, producer-consumer systems and fixed-size storage.

## Deque

Sliding-window processing, task structures requiring access at both ends.

## Priority Queue

CPU/job scheduling, event simulation and service based on urgency.